

Lite框架源码理解--从API到Kernel

以lite/demo/cxx/mobile_light/mobilenetv1_light_api.cc为例，理解lite轻量推理引擎LightPredictor的工作过程。

首先对命令行参数进行解析，必须满足argc=1（使用默认参数）或者argc>=9（传入全部参数）

```
if (argc > 2 && argc < 9) {
    std::cerr
        << "usage: ./" << argv[0] << "\n"
        << "  <naive_buffer_model_dir>\n"
        << "  <raw_input_shapes>, eg: 1,3,224,224 for 1 input; "
        << "    1,3,224,224:1,5 for 2 inputs\n"
        << "  <repeats>, eg: 100\n"
        << "  <warmup>, eg: 10\n"
        << "  <power_mode>, 0: big cluster, high performance\n"
        << "                  1: little cluster\n"
        << "                  2: all cores\n"
        << "                  3: no bind\n"
        << "  <thread_num>, eg: 1 for single thread \n"
        << "  <accelerate_opencv>, this option takes effect only when model "
        << "can be running on opencv backend.\n"
        << "                  0: disable opencv kernel cache & tuning\n"
        << "                  1: enable opencv kernel cache & tuning\n"
        << "  <print_output>, 0: disable print outputs to stdout\n"
        << "                  1: enable print outputs to stdout\n"
        << std::endl;
    return 0;
}
```

参数检查完成后，执行RunModel()函数。

```
RunModel(model_dir,
         input_shapes,
         repeats,
         warmup,
         power_mode,
         thread_num,
         accelerate_opencv,
         print_output_elem);

return 0;
```

首先对预测引擎进行配置，轻量级Predictor通过MobileConfig进行配置。

```

133 // 1. Set MobileConfig
134 MobileConfig config;
135 config.set_model_from_file(model_dir);

```

对当前部署环境进行检测，确定是在CPU还是在GPU上执行。接着根据MobileConfig信息创建PaddlePredictor对象。

```

195 config.set_power_mode(static_cast<paddle::lite_api::PowerMode>(power_mode));
196 config.set_threads(thread_num);
197 // 2. Create PaddlePredictor by MobileConfig
198 std::shared_ptr<PaddlePredictor> predictor =
199     CreatePaddlePredictor<MobileConfig>(config);

```

需要注意这里是用父类（PaddlePredictor）的指针指向了子类（LightPredictor）的对象。

```

177 template <>
178 std::shared_ptr<PaddlePredictor> CreatePaddlePredictor(
179     const MobileConfig& config) {
180     auto x = std::make_shared<lite::LightPredictorImpl>();
181     x->Init(config);
182     return x;

```

```

155 class LightPredictorImpl : public lite_api::PaddlePredictor {
156 > public: ...
188 private:
189     std::unique_ptr<lite::LightPredictor> raw_predictor_;
190 };

```

接着调用x->Init(config)接口函数对Predictor进行初始化。

```

35 void LightPredictorImpl::Init(const lite_api::MobileConfig& config) {
36     // LightPredictor Only support NaiveBuffer backend in publish lib
37     if (config.lite_model_file().empty()) {
38 >     raw_predictor_.reset(new LightPredictor( ...
46     } else {
47     raw_predictor_.reset(new LightPredictor(      津, 上个月 • support runtime r
48         config.lite_model_file(),
49         config.is_model_from_memory(),
50         (config.precision_mode() == lite_api::LITE_PRECISION_LOW) ? true
51         : false));
52     }

```

首先对LightPredictor对象进行实例化。

```

43 // constructor function of LightPredictor, `lite_model_file` refers to data in
44 // model file or buffer, `model_from_memory` refers to whther to load model
45 // from memory.
46 LightPredictor(const std::string& lite_model_file, huzhiqiang, 3年前 • 【M
47 |         |         |         |         |         |         |         |         |         |
48 |         |         |         |         |         |         |         |         |         |
49 use_low_precision_ = use_low_precision;
50 scope_ = std::make_shared<Scope>();
51 program_desc_ = std::make_shared<cpp::ProgramDesc>();
52 Build(lite_model_file, model_from_memory);
53 }

```

其中Scope保存了所有weights的数值信息，ProgramDesc保存了所有op和var（Tensor）的信息。

```

37 /*
38  * The light weight predictor, mainly for mobile. It loads an optimized model,
39  * and will not depend on the MIR or perform latter optimization.
40  */
41 class LITE_API LightPredictor {
42 > public: | Yan Chunwei, 3年前 • publish lite (#1800) ... ..
106 > #ifdef LITE_WITH_METAL ...
114 #endif
115
116 > private: ...
138 > #ifdef ENABLE_ARM_FP16 ...
140 #endif
141
142 > void ClearTensorArray( ...
145 private:
146     std::shared_ptr<Scope> scope_;
147     std::unique_ptr<RuntimeProgram> program_;
148     std::shared_ptr<cpp::ProgramDesc> program_desc_;
149     std::vector<std::string> input_names_;
150     std::vector<std::string> output_names_;
151     std::vector<PrecisionType> input_precisions_;
152     bool bool_clear_tensor_ = false;
153 };
154

```

声明后，调用Build接口对模型文件进行加载，进而将scope和programdesc进行实例化。

```

25 void LightPredictor::Build(const std::string& lite_model_file,
26 |         |         |         |         |         |         |         |         |         |
27 |         |         |         |         |         |         |         |         |         |
28 LoadModelNaiveFromMemory(
29 |         |         |         |         |         |         |         |         |         |
30 } else {
31 LoadModelNaiveFromFile(lite_model_file, scope_.get(), program_desc_.get());
32 }

```

```
756  /*
757  * Binary structure of naive_buffer model: model.nb
758  * -----
759  * |      |      PART      |      Precision      |      Length(byte)      |
760  * |  1  | meta_version | uint16_t      |      2      |
761  * |  2  | opt_version  | char[16]     |      16     |
762  * |  3  | topo_size    | uint64_t     |      8      |
763  * |  4  | topo_data    | char[]       | topo_size byte |
764  * |  5  | param_data   | char[]       |              |
765  * -----
766  * Meaning of each part:
767  *   meta_version: meata_version, 0 default.
768  *   opt_version:  lite_version of opt tool that transformed this model.
769  *   topo_size:   length of `topo_data'.
770  *   topo_data:    contains model's topology data.
771  *   param_data:   contains model's params data.
772  */
773  💡
774  > void LoadModelNaiveFromFile(const std::string &filename, huzhiqiang, 3年前
```

```

774 void LoadModelNaiveFromFile(const std::string &filename,
775                               Scope *scope,
776                               cpp::ProgramDesc *cpp_prog) {
777     CHECK(cpp_prog);
778     CHECK(scope);
779     // ModelFile
780     const std::string prog_path = filename;
781     // Offset
782     model_parser::BinaryFileReader reader(filename, 0);
783
784     // (1) get meta version
785     uint16_t meta_version;
786     reader.Read(&meta_version, sizeof(uint16_t));
787     VLOG(4) << "Meta_version:" << meta_version;
788
789     switch (meta_version) {
790     case 0:
791 #ifndef LITE_ON_TINY_PUBLISH
792         LoadModelNaiveV0FromFile(filename, scope, cpp_prog);
793 #else
794         LOG(FATAL) << "Paddle-Lite v2.7 has upgraded the naive-buffer model "
795                     << "format. Please use the OPT to generate a new model. "
796                     << "Thanks!";
797 #endif
798         break;
799     case 1:
800         LoadModelFbsFromFile(&reader, scope, cpp_prog, 1);
801         break;
802     case 2:
803         LoadModelFbsFromFile(&reader, scope, cpp_prog, 2);
804         break;
805     default:
806         LOG(FATAL) << "The model format cannot be recognized. Please make sure "
807                     << "you use the correct interface and model file.";
808     }
809 }

```

对于 (meta_version=0) 的模型文件，不能使用tiny_publish编译选项。如果使用了tiny_publish选项，则必须使用经过opt工具转换后的模型文件，即naive_buffer格式的模型文件 (meta_verssion=1或者2)。

i 此处理解为：通过tiny_publish选项隔离旧格式模型文件的推理过程，减小编译产物的体积。

对于flatbuffer格式的模型文件，也就是经过opt工具转换后的模型文件。使用LoadModelFbsFromFile()接口进行模型加载。

首先读取flatbuffer格式模型的paddle版本和opt版本，检查两者是否一致。

接着获取topo_size信息，接着将topo_size byte的topo_data信息读入cpp::ProgramDesc中。

```

897 #ifdef LITE_ON_FLATBUFFERS_DESC_VIEW
898     lite::model_parser::Buffer buf(topo_size);
899     reader->Read(buf.data(), topo_size);
900     cpp_prog->Init(std::move(buf));
901 #elif LITE_ON_TINY_PUBLISH
902     LOG(FATAL) << "Since no data structure of Flatbuffers has been constructed, "
903     "the model cannot be loaded.";

```

而LITE_ON_FLATBUFFERS_DESC_VIEW宏定义是在编译时与TINY_PUBLISH绑定在一起的。

```

279 if (LITE_ON_TINY_PUBLISH)
280     add_definitions("-DLITE_ON_TINY_PUBLISH")
281     add_definitions("-DLITE_ON_FLATBUFFERS_DESC_VIEW")
282     add_definitions("-DLITE_WITH_FLATBUFFERS_DESC")
283     message(STATUS "Flatbuffers will be used as cpp default program description.")
284 else()
285     add_definitions("-DLITE_WITH_FLATBUFFERS_DESC")
286 endif()

```

cmake/configure.cmake

记着读取模型文件剩余部分，即params部分。

```

911 /* 2. Load scope from params.fbs */
912 switch (meta_version) {
913     case 1: {
914         /* load scope from param.fbs with meta_version=1 */
915         lite::model_parser::Buffer buf(reader->length() - reader->current());
916         reader->Read(buf.data(), reader->length() - reader->current());
917         fbs::CombinedParamsDescView params(std::move(buf));
918         fbs::deprecated::SetScopeWithCombinedParams(scope, params);
919         break;
920     }
921     case 2: {
922         /* load scope from param.fbs with meta_version=2 */
923         fbs::ParamDeserializer deserializer(reader);
924         deserializer.ForwardRead(scope);
925         break;
926     }
927     default:
928         LOG(FATAL) << "Unspported model meta_version " << meta_version;
929         break;
930 }

```

至此，模型文件加载完成，topo信息（op和var）保存在cpp::ProgramDesc对象中，params信息（weights）保存在Scope对象中。并且完成了对LightPredictor对象中相应数据成员的初始化。

- 接着，对压缩weights（int8/int16）和量化weights（float16）进行运行时处理（对于前者，还原成float32进行处理；对于后者，运行前其数值被量化，但是其存储格式仍为float32，需要在运行时将其转换为float16存储）。

```

34 // For weight quantization of post training, load the int8/16 weights
35 // for optimized model, and dequant it to fp32.
36 DequantizeWeight();
37 #ifdef ENABLE_ARM_FP16
38 // fp16 Weight convert
39 WeightFP32ToFP16();
40 #endif

```

对weights转换完成后，通过topo信息（program_desc_指针）构建出LightPredictor运行时的主体RuntimeProgram对象。

```

41 BuildRuntimeProgram(program_desc_, use_low_precision_);
42 PrepareFeedFetch();
43 }

```

BuildRuntimeProgram()首先在scope中为Block中的临时var变量（临时tensor）分配空间（首先分配了feed和fetch）。对于block中的op，针对tensorArray历史遗留变量类型做了标记处理。

```

196 void LightPredictor::BuildRuntimeProgram(
197     const std::shared_ptr<const cpp::ProgramDesc>& program_desc,
198     bool use_precision_low) {
199     auto* exe_scope = &scope_>NewScope();
200     // Prepare workspace
201     scope_>Var("feed")>GetMutable<std::vector<lite::Tensor>>();
202     scope_>Var("fetch")>GetMutable<std::vector<lite::Tensor>>();
203     CHECK(program_desc);
204     auto block_size = program_desc->BlocksSize();
205     CHECK(block_size);
206     for (size_t block_idx = 0; block_idx < block_size; ++block_idx) {
207         auto block_desc = program_desc->GetBlock<cpp::BlockDesc>(block_idx);
208         auto var_size = block_desc->VarsSize();
209         for (size_t var_idx = 0; var_idx < var_size; ++var_idx) {
210             auto var_desc = block_desc->GetVar<cpp::VarDesc>(var_idx);
211             if (!var_desc->Persistable()) {
212                 auto* var = exe_scope->Var(var_desc->Name());
213                 if (var_desc->GetType() == lite::VarDescAPI::Type::LOD_TENSOR) {
214                     const auto var_data_type =
215                         ConvertPrecisionType(var_desc->GetDataType());
216                     auto* tensor = var->GetMutable<lite::Tensor>();
217                     tensor->set_precision(var_data_type);
218                 }
219             } else {
220                 if (var_desc->Name() == "feed" || var_desc->Name() == "fetch") continue;
221                 scope_>Var(var_desc->Name());
222             }
223         }
224         auto op_size = block_desc->OpsSize();
225         for (size_t op_idx = 0; op_idx < op_size; ++op_idx) {
226             auto op_desc = block_desc->GetOp<cpp::OpDesc>(op_idx);
227             if (op_desc->Type() == "lod_array_length") bool_clear_tensor_ = true;
228         }
229     }
230 }

```

```

44 Variable *Scope::Var(const std::string &name) { Yan Chunwei,
45     SCOPE_VARS_WRITER_LOCK
46     auto *var = FindVar(name);
47     if (var) return var;
48     // create a new variable.
49     vars_.emplace(name, std::unique_ptr<Variable>(new Variable));
50     return vars_[name].get();
51 }

```

接着是运行时fp16+fp32混合精度的处理，对于block中的每一个op，如果该op已经被分配了kernel，解析出kernel对应的Place信息，当kernel对应的precision为float或any时，且当前设备支持fp16推理，则将precision改为KFP16。并通过op->CreateKernels接口来更新该op对应的kernel（pick_kernel()接口完成kernel的替换）。至此，对program_desc和scope的处理工作完成，最后实例化RuntimeProgram对象，只传入根block节点。

```

287 // Only extracting the ops and generate the runtime program from the main
288 // block desc
289 program_.reset(new RuntimeProgram(
290     program_desc, exe_scope, kRootBlockIdx, use_low_precision_));
291 }

```

RuntimeProgram的构造函数主要由一个主循环组成，每次迭代在instructions_数组中添加一项，为一个op和对应kernel组成的pair，也就是说，在运行时，op和kernel被看成是一条指令。

每次迭代首先为op添加scope中的tensor信息，接着对op对应的kernel做处理，如果是fp16推理，需要将kernel对应的tensor信息进行设置，同时在需要的位置插入calib算子。具体实现参考lite/core/program.cc。

```

271 // Create runtime program from sub_block desc according to block_idx and
272 // program_desc, which is used for while/conditional_block/subgraph op.
273 RuntimeProgram::RuntimeProgram(
274     const std::shared_ptr<const cpp::ProgramDesc>& program_desc,
275     Scope* exec_scope,
276     int block_idx,
277     bool use_precision_low)
278     : exec_scope_(exec_scope) {
279     CHECK(program_desc);

```

最后，调用Init()接口对RuntimeProgram进行初始化。

```

563 }
564 instructions_[kRootBlockIdx].emplace_back(std::move(op), std::move(kernel));
565 }
566 Init();
567 }

```

初始化工作包括：初始化profiler，各类GPU设备的初始化，以及CPU的Context的设置。

```

229 void Init() {
230 >   if (instructions_.empty()) { ...
232   }
233 #ifdef LITE_WITH_PROFILE
234   set_profiler();          石晓伟, 3年前 • refactor profile tools, test=develop (#2536) ...
235 #endif
236 > #ifdef LITE_WITH_NVTX ...
244 #endif
245
246 > #ifdef LITE_WITH_OPENCL ...
255 #endif
256
257   for (auto& inst : instructions_[kRootBlockIdx]) {
258     KernelBase* kernel = inst.mutable_kernel();
259     if (kernel->target() == TARGET(kOpenCL)) {
260 > #if defined(LITE_WITH_OPENCL) ...
271 #endif
272     } else if (kernel->target() == TARGET(kMetal)) {
273 > #if defined(LITE_WITH_METAL) ...
281 #endif
282     } else {
283       if (kernel != nullptr) {
284         kernel->SetContext(
285           ContextScheduler::Global().NewContext(kernel->target()));
286       }
287     }
288   }
289 }

```

接着回到LightPredictorImpl::Init接口，在实例化LightPredictor之后，完成后续，power_mode、threads的设置即可。

回到mobilenetv1_light_api.cc，在实例化predictor之后，对于repeats循环，每次迭代调用predictor->Run()执行具体的推理过程。

```

233   for (size_t ridx = 0; ridx < repeats; ++ridx) {
234     auto start = GetCurrentUS();
235
236     predictor->Run();          Yuan Shuai, 3年前 • [LITE][D
237
238     auto duration = (GetCurrentUS() - start) / 1000.0;
239     sum_duration += duration;
240   }
241 }
242
243 void LightPredictorImpl::Run() {          TianXiaogang, 3年前 •
244 #ifdef LITE_WITH_ARM
245   lite::DeviceInfo::Global().SetRunMode(mode_, threads_);
246 #endif
247   raw_predictor_->Run();
248 }

```

```

69 void Run() {
70     CheckInputValid();
71     program_>Run();
72     if (bool_clear_tensor_) ClearTensorArray(program_desc_);
73 }

```

```

592 void RuntimeProgram::Run() {
593     #ifdef LITE_WITH_PRECISION_PROFILE
594         auto inst_precision_profiler = paddle::lite::profile::PrecisionProfiler();
595         std::string precision_profiler_summary =
596             inst_precision_profiler.GetSummaryHeader();
597     #endif
598
599     #ifdef LITE_WITH_NVTX
606     #endif
607
608     #ifdef LITE_WITH_FPGA
611     #endif
612
613     int idx = -1;
614
615     auto& insts = instructions_[kRootBlockIdx];
616     for (auto& inst : insts) {
617         ++idx;
618         #if !defined(LITE_WITH_FPGA) && !defined(LITE_WITH_METAL)
619             if (inst.is_feed_fetch_op()) continue;
620         #endif
621         #ifdef LITE_WITH_NVTX
627         #endif
628         #ifdef LITE_WITH_CUDA
632         #endif
633
634         #ifdef LITE_WITH_FPGA
636         #endif
637
638         #ifdef LITE_WITH_OPENCL
641         #endif
642
643         inst.Run();
644     }

```

```
822 void Instruction::Run() { Yan Chunwei, 3年前 • publish lite (#1800) ...
823 #ifdef LITE_WITH_PROFILE
824     CHECK(profiler_) << "Profiler pointer of kernel can not be nullptr. "
825     | | | | | | | | "When LITE_WITH_PROFILE is defined, please set a "
826     | | | | | | | | "Profiler for Instruction.";
827     profiler_>StartTiming(
828     | | profile::Type::kCreate, profile_id_, kernel_>mutable_context());
829 #endif
830     CHECK(op_) << "op null";
831     CHECK(kernel_) << "kernel null";
832
833     if (first_epoch_) {
834         first_epoch_ = false;
835         CHECK(op_>CheckShape());
836     }
837
838     if (op_>run_once() && has_run_) {
839         return;
840     }
841
842     op_>InferShape();
843     kernel_>Launch();
844     has_run_ = true;
845
846 #ifdef LITE_WITH_PROFILE
847     if (first_epoch_for_profiler_) {
848         kernel_>SetIsKernelTest(false);
849         auto* op_ch = profiler_>GetOpCharacter(profile_id_);
850         SetProfileRuntimeOpInfo(op_ch);
851         first_epoch_for_profiler_ = false;
852     }
853 #endif
854 }
```

```

84 void Launch() { Yan Chunwei, 3年前 • publish lite (#1800) ...
85     /// First run, init kernel, do weights transform once
86     if (is_first_epoch_) {
87         PrepareForRun();
88         is_first_epoch_ = false;
89     }
90     /// re-init the kernel if needed (input shape should be checked in conv
91     /// kernel)
92     ReInitWhenNeeded();
93
94     // Reset the workspace to make every kernel in the same thread to share the
95     // temporary memory.
96     Workspace::Global_Host().AllocReset();
97 #if defined(LITE_WITH_X86)
98     Workspace::Global_X86().AllocReset();
99 #endif
100 #if defined(LITE_WITH_CUDA)
101     Workspace::Global_CUDA().AllocReset();
102 #endif
103 #if defined(LITE_WITH_METAL)
104     Workspace::Global_METAL().AllocReset();
105 #endif
106 #if defined(LITE_WITH_MLU)
107     Workspace::Global_MLU().AllocReset();
108 #endif
109
110 #ifdef LITE_WITH_PROFILE
111     if (!is_kernel_test_) {
112         profiler_>StopTiming(profile::Type::kCreate, profile_id_, ctx_.get());
113         profiler_>StartTiming(profile::Type::kDispatch, profile_id_, ctx_.get());
114     }
115
116     Run();

```

```

231 template <TargetType Target,
232           PrecisionType Precision,
233           DataLayoutType DataLayout = DataLayoutType::kNCHW>
234 class KernelLite : public KernelBase {
235 public:
236     /// Run the kernel.
237     virtual void Run() override { CHECK(false) << "Not Implemented"; }
238
239     TargetType target() const override { return Target; }
240     PrecisionType precision() const override { return Precision; }
241     DataLayoutType layout() const override { return DataLayout; }
242     Place place() const override { return Place{Target, Precision, DataLayout}; }
243     std::string name() const override;
244
245     void Touch() {}
246
247     KernelLite() = default;
248     virtual ~KernelLite() = default;
249 };

```

具体的Run函数在每个kernel实现中重载。